

Zde je detailní studijní materiál k okruhu "Kombinační logické obvody", doplněný a naformátovaný dle vašich přísných pravidel.

34. Kombinační logické obvody

Klíčové pojmy

- Kombinační logický obvod
- Logické hradlo
- Booleova algebra
- Pravdivostní tabulka
- Minterm
- Maxterm
- Úplná disjunktivní normální forma (UDNF)
- Úplná konjunktivní normální forma (UKNF)
- Karnaughova mapa (K-mapa)
- Propagační zpoždění
- Univerzální logický člen
- Multiplexor (MUX)
- Demultiplexor (DEMUX)
- Dekodér (DEC)
- Kodér (ENC)
- Sčítačka
- Komparátor

A. Charakteristika kombinačních logických obvodů

Kombinační logický obvod je digitální obvod, jehož výstupní hodnoty (stavy) jsou v libovolném časovém okamžiku závislé výhradně na aktuálních

hodnotách jeho vstupů v tomto konkrétním okamžiku. Obvod nemá žádnou vnitřní paměť, která by uchovávala informace o předchozích stavech vstupů.

Hlavní rysy:

- **Bezpečnost:** Výstupy obvodu jsou funkcí pouze aktuálních vstupů. Nezajímají ho předchozí vstupy ani vnitřní stavy.
- **Absence zpětné vazby:** Neobsahuje žádné zpětnovazební smyčky, které by vracely výstupy zpět na vstupy a vytvářely tak paměť.
- **Matematický popis:** Chování obvodu lze jednoznačně popsat soustavou Booleových algebraických funkcí nebo pravdivostní tabulkou.
- **Propagační zpoždění:** Po změně vstupů se výstupy změní po velmi krátkém, ale nenulovém čase, který je dán fyzikálními vlastnostmi křemíku a počtem hradel v cestě signálu.
- **Příklady v praxi:**
 - Sčítačka (např. v ALU procesoru)
 - Multiplexor (výběr dat)
 - Demultiplexor (distribuce dat)
 - Kodér (převod z desítkové na binární)
 - Dekodér (převod z binární na desítkovou, adresování paměti)
 - Komparátor (porovnávání čísel)

Pro kontrast u státnic: Opakem jsou **sekvenční logické obvody** (např. klopné obvody flip-flop, registry, čítače), jejichž výstup závisí nejen na současných vstupech, ale i na předchozím stavu (mají paměť) a často jsou řízeny hodinovým signálem.

B. Realizace pomocí logických hradel

Logická hradla jsou základní fyzické stavební prvky (integrované obvody), které realizují elementární Booleovy operace.

Základní logická hradla:

- **AND (logický součin):** Výstup je 1, jen když jsou všechny vstupy 1.
 - Funkce: $Y = A \cdot B$

- Pravdivostní tabulka:

A	B	Y
0	0	0
0	1	0
1	0	1
1	1	1

- **OR (logický součet):** Výstup je 1, když je alespoň jeden vstup 1.

- Funkce: $Y = A + B$
- Pravdivostní tabulka:

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

- **NOT (negace/invertor):** Výstup je opačný než vstup.

- Funkce: $Y = \overline{A}$
- Pravdivostní tabulka:

A	Y
0	1
1	0

- **NAND (NOT AND):** Výstup je 0, jen když jsou všechny vstupy 1.

- Funkce: $Y = \overline{A \cdot B}$

- **NOR (NOT OR):** Výstup je 1, jen když jsou všechny vstupy 0.

- Funkce: $Y = \overline{A + B}$

- **XOR (exkluzivní OR):** Výstup je 1, když jsou vstupy různé.

- Funkce: $Y = A \oplus B = A \cdot \overline{B} + \overline{A} \cdot B$

- **XNOR (exkluzivní NOR):** Výstup je 1, když jsou vstupy stejné.

- Funkce: $Y = \overline{A \oplus B} = A \cdot B + \overline{A} \cdot \overline{B}$

Postup návrhu obvodu z hradel (od zadání k realizaci):

1. **Sestavení pravdivostní tabulky:** Na základě slovního zadání (nebo specifikace) určíme všechny možné kombinace vstupů a odpovídající požadované výstupy (0 nebo 1).

- *Příklad pro XOR (2 vstupy A, B, 1 výstup Y):*

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

2. **Zápis Booleovy funkce:** Z pravdivostní tabulky zapíšeme logickou funkci. Nejčastěji se používá **Úplná disjunktivní normální forma**

(UDNF), která je součtem (OR) všech tzv. **mintermů** (řádků tabulky), kde je výstup 1. Každý minterm je součinem (AND) vstupních proměnných, přičemž proměnná je negovaná, pokud má v daném řádku hodnotu 0.

- Pro XOR z příkladu:

- Řádek 2 ($A = 0, B = 1, Y = 1$): minterm je $\bar{A} \cdot B$
- Řádek 3 ($A = 1, B = 0, Y = 1$): minterm je $A \cdot \bar{B}$
- Výsledná funkce v UDNF: $Y = \bar{A} \cdot B + A \cdot \bar{B}$

3. **Minimalizace funkce:** Cílem je zjednodušit Booleovu funkci, aby se snížil počet potřebných hradel a tím i náklady, spotřeba energie a propagační zpoždění.

- **Booleova algebra:** Používají se zákony Booleovy algebry (např. distributivní zákon $A \cdot (B + C) = A \cdot B + A \cdot C$, absorpční zákon $A + A \cdot B = A$, De Morganovy zákony $\overline{A \cdot B} = \bar{A} + \bar{B}$ a $\overline{A + B} = \bar{A} \cdot \bar{B}$).
- **Karnaughova mapa (K-mapa):** Pro funkce s menším počtem proměnných (do 4-5) je to grafická metoda, která umožňuje vizuálně identifikovat a seskupit sousední mintermy, a tím je zjednodušit.

4. **Nakreslení schématu:** Ze zjednodušeného Booleova výrazu poskládáme výslednou síť logických hradel.

Univerzálnost hradel NAND / NOR

U zkoušky je dobré zmínit, že hradla NAND a NOR jsou tzv. **univerzální logické členy**. Pomocí samotných hradel NAND (nebo samotných NOR) dokážeme poskládat jakoukoliv jinou logickou funkci (NOT, AND, OR, XOR atd.). V praxi je výroba čipů z jednoho typu hradla často levnější a efektivnější.

- **Příklad - NOT z NAND:**

- $A \text{ NAND } A = \overline{A \cdot A} = \bar{A}$
- Stačí spojit oba vstupy hradla NAND dohromady.

C. Realizace pomocí Multiplexorů (MUX)

Multiplexor (MUX) je kombinační obvod, který funguje jako digitální přepínač. Má 2^n datových vstupů ($D_0, D_1, \dots, D_{2^n-1}$), n selekčních (adresových) vstupů ($S_0, S_1, \dots, S_{n-1}$) a jeden jediný výstup (Y). Podle binárního čísla přivedeného na selekční vstupy se příslušný datový vstup propojí s výstupem.

- **Funkce MUX:** Výstup Y je roven datovému vstupu, jehož index odpovídá binární hodnotě selekčních vstupů.
 - Například pro MUX 4:1 (2 selekční vstupy S_1, S_0 a 4 datové vstupy D_0, D_1, D_2, D_3):
$$Y = \overline{S_1} \cdot \overline{S_0} \cdot D_0 + \overline{S_1} \cdot S_0 \cdot D_1 + S_1 \cdot \overline{S_0} \cdot D_2 + S_1 \cdot S_0 \cdot D_3$$

Jak pomocí MUX realizovat libovolnou logickou funkci:

Multiplexor je geniální v tom, že dokáže realizovat jakoukoliv Booleovu funkci o n proměnných bez nutnosti minimalizace kódu nebo propojování složitých hradel.

Postup:

1. **Připojení proměnných na selekční vstupy:** Proměnné naší funkce (např. A, B) připojíme přímo na selekční (adresové) vstupy multiplexoru. Pokud má funkce n proměnných, použijeme multiplexor s n selekčními vstupy (tedy 2^n datovými vstupy).
2. **Přivedení logických hodnot na datové vstupy:** Na datové vstupy (D_0 až D_{2^n-1}) přivedeme napevno logickou 0 (GND / zem) nebo logickou 1 (VCC / napájení) přesně podle toho, co pro danou kombinaci selekčních vstupů vyžaduje pravdivostní tabulka funkce.
3. **Příklad: Realizace funkce XOR se dvěma vstupy A a B pomocí MUX 4:1.**
 - Pravdivostní tabulka XOR:

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0
 - Použijeme multiplexor 4:1 (má 2 selekční vstupy).

- Na selekční vstupy připojíme A a B (např. $S_1 = A, S_0 = B$).
- Na datové vstupy D_0 až D_3 přivedeme postupně hodnoty z výstupu Y pravdivostní tabulky XOR, které odpovídají kombinacím AB :
 - $AB = 00 \rightarrow D_0 = 0$
 - $AB = 01 \rightarrow D_1 = 1$
 - $AB = 10 \rightarrow D_2 = 1$
 - $AB = 11 \rightarrow D_3 = 0$
- Obvod je hotový.

D. Realizace pomocí Dekodérů (DEC)

Dekodér (konkrétně n -bitový dekodér na 2^n linek, např. dekodér z 3 do 8) je kombinační obvod, který má n vstupů a 2^n výstupů. Když na vstup přivedeme binární číslo, aktivuje se (přepne se do logické 1, pokud je aktivní high, nebo do 0, pokud je aktivní low) právě ten jeden výstup, jehož index odpovídá zadanému binárnímu číslu. Všechny ostatní výstupy jsou neaktivní.

- **Funkce DEC:** Pro n vstupů $A_{n-1}, \dots, A_0$ a 2^n výstupů $Y_0, \dots, Y_{2^n-1}$, se aktivuje pouze jeden výstup Y_k , kde k je dekadická hodnota binárního čísla na vstupech. Každý výstup Y_k odpovídá jednomu **mintermu** vstupních proměnných.
 - *Příklad pro dekodér 2:4 (vstupy A_1, A_0 , výstupy Y_0, Y_1, Y_2, Y_3):*
 - $Y_0 = \overline{A_1} \cdot \overline{A_0}$
 - $Y_1 = \overline{A_1} \cdot A_0$
 - $Y_2 = A_1 \cdot \overline{A_0}$
 - $Y_3 = A_1 \cdot A_0$

Jak pomocí DEC realizovat libovolnou logickou funkci:

Každý jednotlivý výstup dekodéru reprezentuje jeden konkrétní minterm (jeden konkrétní řádek pravdivostní tabulky, kde je výstup 1).

Postup:

1. **Připojení vstupů na dekodér:** Vstupy naší funkce připojíme na vstupy dekodéru. Pokud má funkce n proměnných, použijeme dekodér s n

vstupy (a 2^n výstupy).

2. **Identifikace mintermů:** V pravdivostní tabulce si najdeme všechny řádky, kde má být výstup funkce 1.
3. **Zapojení hradla OR:** Výstupy dekodéru, které odpovídají těmto mintermům (řádkům s výstupem 1), jednoduše vyvedeme ven a zapojíme je do jednoho společného vícevstupového hradla OR. Výstup z tohoto hradla OR je výsledkem naší funkce.
4. **Příklad: Realizace funkce XOR se dvěma vstupy A a B pomocí dekodéru 2:4 a hradla OR.**

- Pravdivostní tabulka XOR:

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0
- Vstupy A , B připojíme na vstupy dekodéru.
- Výstup Y je 1 pro kombinace $AB = 01$ a $AB = 10$.
- Tyto kombinace odpovídají výstupům dekodéru Y_1 a Y_2 .
- Výsledná funkce: $Y = Y_1 + Y_2 = (\bar{A} \cdot B) + (A \cdot \bar{B})$
- Zapojíme výstupy Y_1 a Y_2 dekodéru do dvouvstupového hradla OR. Výstup hradla OR je naše funkce XOR.

Shrnutí možností realizace pro zkoušejícího:

- **Hradla:**
 - **Výhody:** Nejzákladnější a nejflexibilnější pro jednoduché funkce. Umožňuje optimalizaci na nejnižší úrovni.
 - **Nevýhody:** Vyžaduje minimalizaci (Karnaughovy mapy nebo Booleova algebra), což může být pro složitější funkce časově náročné a náchylné k chybám. Výsledkem je zakázkový, specifický obvod.
- **Multiplexor:**
 - **Výhody:** Univerzální přístup, tabulka se přímo "nadrátuje" na datové vstupy. Ideální pro rychlý návrh bez minimalizace. Jednoduchá změna funkce pouhou změnou vstupních hodnot.
 - **Nevýhody:** Pro mnoho vstupních proměnných může být potřeba velký MUX s mnoha datovými vstupy, což může být

neefektivní z hlediska počtu tranzistorů.

- **Dekodér:**
 - **Výhody:** Skvělý v případě, kdy potřebujeme z jedněch vstupů generovat více různých funkcí současně (např. řízení segmentového displeje, dekodér instrukcí v procesoru, adresování paměti). Stačí jeden dekodér a pro každou funkci jen přidat příslušné hradlo OR.
 - **Nevýhody:** Pro realizaci jedné funkce může být méně efektivní než MUX nebo minimalizovaná hradlová síť, protože generuje všechny mintermy, i ty nepotřebné.

Praktické použití

- **Aritmeticko-logické jednotky (ALU):** Sčítačky, komparátory, logické operace (AND, OR, XOR) jsou základními stavebními kameny ALU v procesorech.
- **Adresování paměti:** Dekodéry se používají k výběru konkrétní paměťové buňky nebo bloku paměti na základě adresy.
- **Datové směrování:** Multiplexory se používají k výběru jednoho z mnoha datových zdrojů a jeho směrování na jeden výstup (např. výběr registru v procesoru). Demultiplexory pak k distribuci jednoho datového vstupu na jeden z mnoha výstupů.
- **Řízení displejů:** Dekodéry se používají k převodu binárních čísel na signály pro řízení segmentů sedmisegmentových displejů nebo pixelů složitějších displejů.
- **Kódování/dekódování:** Převod mezi různými číselnými soustavami nebo formáty (např. BCD na 7-segmentový kód).
- **Řadiče:** V jednoduchých digitálních systémech mohou kombinační obvody realizovat řídicí logiku na základě aktuálních vstupů.

Nejčastější chyby u zkoušky

- **Přidání paměti do kombinačního obvodu:** Záměna s principem sekvenčních obvodů. Kombinační obvod nesmí mít žádné paměťové prvky ani zpětnou vazbu.

- **Chyby v De Morganových zákonech:** Nesprávná aplikace pro minimalizaci funkcí.
- **Neúplná pravdivostní tabulka:** Chybějící kombinace vstupů nebo nesprávné výstupy.
- **Neznalost univerzálnosti NAND/NOR hradel:** Neumět ukázat, jak z nich sestavit základní hradla.
- **Nepochopení principu MUX/DEC pro realizaci funkcí:** Mechanické zapamatování bez pochopení, jak se mapuje pravdivostní tabulka na vstupy.

Typické zkuškové otázky

1. Co je kombinační obvod a jak se liší od sekvenčního obvodu?

- **Odpověď:** Kombinační logický obvod je digitální obvod, jehož výstupní hodnoty jsou v libovolném časovém okamžiku závislé výhradně na aktuálních hodnotách jeho vstupů. Nemá žádnou paměť a neobsahuje zpětnovazební smyčky. Naopak sekvenční logický obvod má paměť, takže jeho výstup závisí nejen na aktuálních vstupech, ale i na předchozím stavu (historii vstupů). Sekvenční obvody často používají hodinový signál pro synchronizaci změn stavu.

2. Popište základní logická hradla a uveďte jejich Booleovy funkce a pravdivostní tabulky.

- **Odpověď:** Základní logická hradla jsou AND ($Y = A \cdot B$), OR ($Y = A + B$), NOT ($Y = \overline{A}$), NAND ($Y = \overline{A \cdot B}$), NOR ($Y = \overline{A + B}$), XOR ($Y = A \oplus B$) a XNOR ($Y = \overline{A \oplus B}$). Pro každé hradlo je nutné uvést jeho pravdivostní tabulku, která pro všechny kombinace vstupů ukazuje odpovídající výstup. (Viz sekce B pro konkrétní tabulky a funkce).

3. Jak se navrhuje kombinační obvod z pravdivostní tabulky a jakou roli hraje minimalizace?

- **Odpověď:** Návrh kombinačního obvodu začíná sestavením pravdivostní tabulky na základě specifikace funkce. Z tabulky se poté zapíše Booleova funkce, nejčastěji v Úplné disjunktivní normální formě (UDNF), kde každý řádek s výstupem 1 je reprezentován mintermem. Následuje fáze minimalizace funkce, která je klíčová pro zjednodušení obvodu. Minimalizace (pomocí Booleovy algebry nebo Karnaughových map) snižuje počet potřebných logických hradel, což vede k nižším nákladům, menší spotřebě energie a kratšímu propagačnímu zpoždění. Z minimalizované funkce se pak nakreslí finální schéma obvodu.

4. Vysvětlete, jak lze libovolnou logickou funkci realizovat pomocí multiplexoru nebo dekodéru.

- **Odpověď:**
 - **Multiplexor (MUX):** Pro realizaci funkce s n proměnnými se použije MUX s n selekčními vstupy (a 2^n datovými vstupy). Vstupní proměnné funkce se připojí na selekční vstupy MUX. Na každý datový vstup MUX (D_0 až D_{2^n-1}) se pak přivede pevná logická hodnota (0 nebo 1) odpovídající výstupu funkce v pravdivostní tabulce pro danou kombinaci selekčních vstupů. Výstup MUX je pak výstupem realizované funkce.
 - **Dekodér (DEC):** Pro realizaci funkce s n proměnnými se použije dekodér s n vstupy (a 2^n výstupy). Vstupní proměnné funkce se připojí na vstupy dekodéru. Každý výstup dekodéru odpovídá jednomu mintermu. Všechny výstupy dekodéru, které odpovídají mintermům, pro které má být výstup funkce 1, se zapojí do vícevstupového hradla OR. Výstup tohoto hradla OR je pak výstupem realizované funkce.

Shrnutí kapitoly

Kombinační logické obvody jsou základními stavebními kameny digitální elektroniky, jejichž výstupy závisí výhradně na aktuálních vstupech, bez

jakékoli paměti. Jejich chování je popsáno Booleovou algebrou a pravdivostními tabulkami. Návrh obvodů z hradel vyžaduje minimalizaci funkcí (např. pomocí Karnaughových map) pro efektivní realizaci. Alternativní a často jednodušší metody realizace funkcí nabízejí multiplexory a dekodéry, které umožňují přímé mapování pravdivostní tabulky na vstupy obvodu. Pochopení těchto principů je klíčové pro návrh a analýzu digitálních systémů, od jednoduchých logických funkcí až po komplexní aritmeticko-logické jednotky v procesorech.

35. Sekvenční logické obvody

Klíčové pojmy

- Sekvenční obvod
- Klopný obvod (Flip-Flop, Latch)
- Synchronní a asynchronní obvody
- Hodiny (CLK)
- Registry
- Posuvný registr
- Čítač
- Konečný automat (FSM)
- Automat typu Moore
- Automat typu Mealy
- Graf stavů
- Tabulka přechodů
- Kódování stavů
- Minimalizace logiky

A. Základní vlastnosti sekvenčních obvodů

Sekvenční logický obvod je digitální obvod, jehož výstupní hodnoty jsou v daném okamžiku závislé nejen na aktuálních hodnotách jeho vstupů, ale také na předchozím vnitřním stavu obvodu (na jeho historii).

Hlavní rysy:

- **Paměť:** Obvod obsahuje paměťové prvky (základním prvkem je klopný obvod), které uchovávají informaci o vnitřním stavu.
- **Zpětná vazba:** Výstupy z paměťových prvků jsou zavedeny zpět do kombinační části obvodu, což umožňuje ovlivnění budoucího stavu na základě stavu minulého.

Rozdělení podle synchronizace:

- **Asynchronní obvody:**
 - Reagují na změnu vstupních signálů okamžitě.
 - Jsou náchylné na hazardní stavy (chyby vlivem různého zpoždění signálů na hradlech), což ztěžuje jejich návrh a spolehlivost.
- **Synchronní obvody (řízené hodinami):**
 - Ke změně vnitřního stavu může dojít pouze při definované změně hodinového signálu CLK (např. na vzestupné nebo sestupné hraně).
 - Celý obvod je tak perfektně časově koordinován, což je standard u moderních procesorů a digitálních systémů.

B. Typy klopných obvodů (Flip-Flops / Latches)

Klopný obvod je základní jednobitový paměťový prvek. Rozlišujeme obvody řízené úrovní signálu (Latch) a obvody řízené hranou signálu (Flip-Flop). Každý má standardně přímý výstup Q a negovaný výstup $\bar{Q}$.

1. Klopný obvod RS (Reset-Set)

- **Vstupy:** S (Set – nastavení do 1), R (Reset – nulování).
 - **Chování:**
 - Když $S = 1, R = 0 \rightarrow Q = 1$.
 - Když $S = 0, R = 1 \rightarrow Q = 0$.
 - Když $S = 0, R = 0$, obvod si pamatuje předchozí stav ($Q_{next} = Q_{prev}$).
 - **Hazardní (zakázaný) stav:** Kombinace $S = 1, R = 1$. V tomto případě by se obvod pokoušel nastavit i nulovat zároveň, výstup by byl nestabilní a nepredikovatelný.
 - **Pravdivostní tabulka (pro RS Latch):**
- | S | R | Q_{next} | $\bar{Q}_{next}$ | Popis |
|-----|-----|------------|------------------|-------------|
| 0 | 0 | Q_{prev} | $\bar{Q}_{prev}$ | Pamatuje si |
| 0 | 1 | 0 | 1 | Reset |
| 1 | 0 | 1 | 0 | Set |
| 1 | 1 | ? | ? | Zakázaný |

2. Klopný obvod D (Data / Delay)

- **Vstupy:** Datový vstup D , hodinový vstup CLK .
- **Chování:** Tento obvod odstraňuje zakázaný stav RS. Funguje tak, že při aktivní hraně hodin CLK jednoduše převezme hodnotu ze vstupu D a vystaví ji na výstup Q . Do té doby si pamatuje předchozí hodnotu bez ohledu na to, jak se vstup D mění. Je to základní prvek registrů.
- **Pravdivostní tabulka (pro D Flip-Flop):**

CLK	D	Q_{next}	Popis
----	--	-----	----
↑	0	0	Reset
↑	1	1	Set
0, 1, ↓	X		

Q_{prev} | Pamatuje si |

3. Klopný obvod JK

- **Vstupy:** J (chová se jako Set), K (chová se jako Reset), CLK .
- **Chování:** Zpřesňuje obvod RS tak, že odstraňuje zakázaný stav.
 - Pokud na vstupy přivedeme $J = 1, K = 0$, obvod při hodinovém impulsu nastaví $Q = 1$.
 - Pokud $J = 0, K = 1$, obvod při hodinovém impulsu nuluje $Q = 0$.
 - Pokud $J = 0, K = 0$, obvod si pamatuje předchozí stav.
 - Pokud $J = 1, K = 1$, obvod při každém hodinovém impulsu zneguje (překlopí) svůj předchozí stav ($Q_{next} = \bar{Q}_{prev}$).
- **Pravdivostní tabulka (pro JK Flip-Flop):**

CLK	J	K	Q_{next}	Popis
----	--	--	-----	----
↑	0	0	Q_{prev}	Pamatuje si
↑	0	1	0	Reset
↑	1	0	1	Set
↑	1	1	$\bar{Q}_{prev}$	Překlopení

4. Klopný obvod T (Toggle)

- **Vstupy:** T (Toggle), CLK .
- **Chování:** Má pouze jeden datový vstup.
 - Pokud $T = 0$, obvod drží stav ($Q_{next} = Q_{prev}$).
 - Pokud $T = 1$, s každou aktivní hranou hodin se stav překlopí ($Q_{next} = \bar{Q}_{prev}$).
- Používá se jako základní stavební prvek binárních čítačů.
- **Pravdivostní tabulka (pro T Flip-Flop):**

CLK	T	Q_{next}	Popis
----	--	-----	----
↑	0	Q_{prev}	Pamatuje si
↑	1	$\bar{Q}_{prev}$	Překlopení

C. Registry a posuvné registry

Sdružením několika klopných obvodů (typicky typu D) vedle sebe do jednoho celku vznikají registry, které dokážou uchovat vícebitové slovo (např. byte, 32-bitové číslo).

1. Paralelní registr (Buffer register)

- Má společný hodinový signál pro všechny klopné obvody.
- Data se zapisují naráz na všech vstupech a jsou naráz dostupná na všech výstupech (PIPO – Parallel In, Parallel Out).
- Slouží k uchování dat v procesoru, například pro dočasné uložení výsledků ALU nebo adres.

2. Posuvný registr (Shift register)

- Klopné obvody jsou zapojeny sériově za sebou (výstup Q prvního obvodu jde do vstupu D druhého).
- S každým hodinovým impulsem se informace posune o jeden bit doprava nebo doleva.
- **Využití:**
 - **Převod dat:** Ze sériového formátu na paralelní (SIPO – Serial In, Parallel Out) nebo naopak (PISO – Parallel In, Serial Out). To je základ fungování komunikačních rozhraní (např. UART, SPI, USB).
 - **Matematické operace:** Posun bitů v registru o jednu pozici doleva odpovídá násobení čísla dvěma. Posun doprava odpovídá celočíselnému dělení dvěma.
 - **Generování sekvencí:** Lze je použít pro generování pseudonáhodných čísel nebo testovacích sekvencí.

D. Čítače (Counters)

Čítače jsou sekvenční obvody určené k počítání hodinových impulsů. Výstupem je binární číslo, které se s každým impulsem inkrementuje (nebo dekrementuje).

1. Asynchronní čítače (Ripple counters)

- Výstup jednoho klopného obvodu (typu T nebo JK v režimu překlápění) slouží jako hodinový vstup pro následující klopný obvod.
- **Nevýhoda:** Zpoždění se sčítá s každým dalším bitem (řetězové zpoždění), pro vysoké frekvence jsou nepoužitelné, protože jednotlivé bity se mění v různých časech.

2. Synchronní čítače

- Všechny klopné obvody jsou řízeny jedním společným hodinovým signálem.
- Změna stavu všech bitů nastane přesně v tentýž okamžik.
- To, který bit se má překlopit, řídí dopředná kombinační logika (např. hradla AND) zapojená mezi obvody, která určuje, zda se má daný klopný obvod překlopit na základě stavů předchozích bitů.

E. Návrh konečných automatů (FSM - Finite State Machine)

Jakýkoliv složitější sekvenční obvod (např. řadič procesoru, řízení křižovatky, protokolový řadič) lze navrhout jako konečný automat. Automat je popsán množinou vstupů, množinou výstupů, množinou vnitřních stavů a přechodovou funkcí.

Rozlišujeme dva základní typy automatů podle toho, jak generují své výstupy:

1. Automat typu Moore

- **Vlastnost:** Výstup automatu je funkcí pouze aktuálního vnitřního stavu.
- Pokud se změní vstup, výstup se nezmění hned, ale změní se až v následujícím taktovém cyklu, kdy automat přejde do nového stavu.
- V grafu stavů se výstup zapisuje přímo dovnitř kroužku stavu (např. Stav / Výstup).

- Bývá jednodušší na časování, protože výstupy jsou stabilní po celou dobu trvání stavu.

2. Automat typu Mealy

- **Vlastnost:** Výstup automatu je funkcí aktuálního vnitřního stavu i a aktuálních vstupů.
- Pokud se změní vstup, výstup může reagovat okamžitě asynchronně v rámci téhož taktu (skrze kombinační logiku), ještě předtím, než se změní samotný stav.
- V grafu stavů se výstup zapisuje na šipku přechodu (např. Vstup / Výstup).
- Má obecně méně stavů než Moore automat pro stejnou funkcionalitu, ale je náchylnější na rušení na vstupech a může generovat krátké "glitch" výstupy při změně vstupů.

Obecný postup návrhu synchronního automatu u zkoušky:

1. Sestavení grafu stavů (nebo tabulky přechodů):

- Specifikujeme, jak automat reaguje na vstupy a jaké stavy vystřídá.
- Definujeme všechny možné stavy, vstupy, výstupy a pravidla pro přechody mezi stavy.

2. Kódování stavů:

- Každému stavu přiřadíme unikátní binární kód.
- Pro N stavů potřebujeme minimálně $\lceil \log_2 N \rceil$ klopných obvodů (např. při 4 stavech potřebujeme 2 klopné obvody: 00, 01, 10, 11).

3. Vytvoření zakódované tabulky přechodů a výstupů:

- Na základě současného stavu (Q_{actual}) a vstupů určíme hodnoty pro budoucí stavy (Q_{next}) a výstupy.
- Tato tabulka slouží jako základ pro návrh kombinační logiky.

4. Minimalizace kombinační logiky:

- Pomocí Karnaughových map (nebo jiných minimalizačních metod) minimalizujeme logické funkce pro:
 - Buzení klopných obvodů (vstupy D nebo J, K paměťových prvků).

- Výstupy automatu.

5. Realizace:

- Zapojíme vypočtená hradla před klopné obvody (pro buzení) a obvod pro výstupy.
- Všechny klopné obvody jsou připojeny ke společnému hodinovému signálu.

Praktické použití

- **Paměťové prvky procesoru:** Registry, program counter, stack pointer.
- **Časovače a čítače:** V mikrokontrolérech, pro generování zpoždění, měření frekvence.
- **Řadiče protokolů:** V komunikačních rozhraních (UART, SPI, I2C, USB, Ethernet).
- **Stavové automaty v FPGA:** Pro implementaci komplexní řídicí logiky a digitálního zpracování signálů.
- **Řízení sekvencí:** Například v automatizaci, řízení výrobních linek.

Nejčastější chyby u zkoušky

- **Zaměňování kombinačního a sekvenčního obvodu.**
- **Ignorování hodin a synchronizace** u synchronních obvodů.
- **Neřešení počátečního stavu** automatu nebo čítače.
- **Neznalost zakázaných stavů** u klopných obvodů (např. RS).
- **Nerozlišování Moore a Mealy automatů** a jejich dopadů na výstupy.

Typické zkouškové otázky

1. Co je sekvenční obvod?

- **Odpověď:** Sekvenční obvod je digitální obvod, jehož výstupní hodnoty v daném okamžiku závisí nejen na aktuálních hodnotách jeho vstupů, ale také na jeho předchozím vnitřním stavu. Obsahuje paměťové prvky (klopné obvody) a zpětnou vazbu.

2. Jak funguje registr a čítač?

- **Odpověď:**
 - **Registr** je skupina klopných obvodů (typicky D-klopných obvodů) zapojených vedle sebe, které uchovávají vícebítové slovo. Všechny klopné obvody v registru jsou řízeny společným hodinovým signálem a data se do nich zapisují nebo z nich čtou paralelně (u paralelního registru) nebo sériově (u posuvného registru).
 - **Čítač** je sekvenční obvod, který počítá hodinové impulsy. Jeho výstupem je binární číslo, které se s každým hodinovým impulsem inkrementuje (nebo dekrementuje). Může být asynchronní (výstup jednoho klopného obvodu budí hodiny dalšího) nebo synchronní (všechny klopné obvody jsou řízeny společnými hodinami a změnu stavu řídí kombinační logika).

3. Jak se popisuje konečný automat?

- **Odpověď:** Konečný automat se popisuje pomocí:
 - **Množiny vstupů:** Všechny možné signály, které automat může přijmout.
 - **Množiny výstupů:** Všechny možné signály, které automat může generovat.
 - **Množiny vnitřních stavů:** Všechny možné konfigurace paměťových prvků automatu.
 - **Přechodové funkce:** Pravidla, která určují, do jakého nového stavu automat přejde na základě aktuálního stavu a vstupů.
 - **Výstupní funkce:** Pravidla, která určují, jaké výstupy automat generuje na základě aktuálního stavu (Moore) nebo aktuálního stavu a vstupů (Mealy).
- Graficky se popisuje **grafem stavů** a formálně **tabulkou přechodů a výstupů**.

Shrnutí kapitoly

Sekvenční logika přidává digitálním obvodům paměť a časovou závislost, což umožňuje realizovat složitější funkce než pouhá kombinační logika. Díky ní lze realizovat paměťové prvky (klopné obvody), registry pro uchování dat, čítače pro počítání událostí a komplexní řídicí jednotky ve formě konečných automatů. Správný návrh sekvenčních obvodů vyžaduje pochopení časování a synchronizace.

36. Aritmetické obvody

Klíčové pojmy

- binární kód
 - dvojkový doplněk
 - znaménkový bit
 - nejvyšší bit (MSB)
 - nejnižší bit (LSB)
 - poloviční sčítačka (Half Adder, HA)
 - úplná sčítačka (Full Adder, FA)
 - sčítačka s řetězovým přenosem (Ripple Carry Adder, RCA)
 - sčítačka s předpovědí přenosu (Carry Look-Ahead Adder, CLA)
 - násobička (Multiplier)
 - sériová násobička
 - paralelní násobička
 - přetečení (Overflow)
 - přenos (Carry)
 - výpůjčka (Borrow)
-

A. Reprezentace čísel v počítači

Abychom mohli navrhnout obvody pro aritmetické operace, musíme nejprve vědět, jak jsou čísla v digitálních systémech kódována.

- **Neznaménková čísla (Unsigned)**
 - Používají přímý binární kód. Všechny bity reprezentují hodnotu.
 - Rozsah pro n bitů je 0 až $2^n - 1$.
- **Znaménková čísla (Signed)**

- Potřebujeme vyjádřit i záporná čísla. Využívají se tři hlavní přístupy:
- **Přímý kód (Znaménko a bity)**
 - Nejvýznamnější bit (MSB – Most Significant Bit) určuje znaménko ($0 = +$, $1 = -$), zbytek bitů je absolutní hodnota.
 - **Nevýhoda:** Má dvě reprezentace nuly ($+0$ a -0), což komplikuje návrh sčítaček.
- **Inverzní kód (Jedničkový doplněk)**
 - Záporné číslo vznikne znegováním všech bitů kladného čísla.
 - **Nevýhoda:** Stále má problém se dvěma nulami ($000\dots 0$ a $111\dots 1$).
- **Doplňkový kód (Dvojkový doplněk)**
 - Záporné číslo vznikne znegováním všech bitů kladného čísla a přičtením jedničky.
 - **Výhoda:** Tento kód v moderních počítačích jednoznačně dominuje, protože odstraňuje dvojí interpretaci nuly a umožňuje provádět sčítání i odčítání pomocí téže sčítačky.
 - Rozsah pro n bitů je -2^{n-1} až $2^{n-1} - 1$.

B. Sčítačky (Adders)

Sčítačka je základním aritmetickým obvodem. Sčítá dvě binární čísla bit po bitu od nejnižšího řádu (LSB – Least Significant Bit) k nejvyššímu.

1. Poloviční sčítačka (HA - Half Adder)

Sčítá dva jednobitové vstupy A a B . Generuje výsledný součet S (Sum) a přenos do vyššího řádu C (Carry Out).

- **Logické rovnice:**

- $S = A \oplus B$ (operace XOR)
- $C = A \cdot B$ (operace AND)

- **Tabulka pravdivosti:**

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

- **Omezení:** Poloviční sčítačku lze použít pouze pro nejnižší bit, protože neumí zpracovat přenos z předchozího (nižšího) řádu.

2. Úplná sčítačka (FA - Full Adder)

Sčítá tři jednobitové vstupy: A , B a přenos z předchozího řádu C_{in} (Carry In). Generuje součet S a nový přenos C_{out} .

- **Logické rovnice:**

- $S = A \oplus B \oplus C_{in}$
- $C_{out} = (A \cdot B) + (C_{in} \cdot (A \oplus B))$

- **Tabulka pravdivosti:**

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0

A	B	C_{in}	S	C_{out}
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

3. Vícebitové sčítačky

- **Sčítačka s řetězovým přenosem (Ripple Carry Adder - RCA):**
 - **Konstrukce:** Vznikne sériovým zapojením n úplných sčítaček za sebou. Výstup přenosu C_{out} z jedné sčítačky se zavede jako vstup C_{in} do sčítačky následující.
 - **Nevýhoda:** Je pomalá. Vyšší bity nemohou spočítat svůj výsledek, dokud k nim nedoputuje (neprožene se celým řetězem hradel) signál přenosu z nejnižších bitů.
 - **Časová složitost:** Roste lineárně s počtem bitů $O(n)$.
- **Sčítačka s předpovědí přenosu (Carry Look-Ahead Adder - CLA):**
 - **Konstrukce:** Odstraňuje řetězové zpoždění. Obsahuje speciální kombinační obvod, který na základě vstupů A a B dokáže okamžitě paralelně "předpovědět" (vypočítat) přenosy pro všechny bity naráz bez čekání na předchozí sčítačky.
 - **Výhoda:** Extrémně rychlá, výpočet probíhá výrazně rychleji než lineární, často se považuje za téměř konstantní pro praktické velikosti, nebo logaritmickou ($O(\log n)$) pro velmi velké N .
 - **Nevýhoda:** Velmi složitá struktura hradel pro větší počet bitů, což vede k vyšší hardwarové náročnosti.

C. Odčítačky (Subtractors) a realizace v doplňkovém kódu

Teoreticky lze navrhnout samostatný obvod úplné odčítačky (pomocí operací s výpůjčkou – Borrow). V praxi se to však z ekonomických a

prostorových důvodů v procesorech nedělá.

Díky vlastnostem doplňkového kódu lze totiž operaci odčítání $A - B$ převést na sčítání s negovaným číslem:

$$A - B = A + (-B) = A + \bar{B} + 1$$

kde $\bar{B}$ je bitová negace (jedničkový doplněk) čísla B .

- **Realizace univerzálního obvodu Sčítačka/Odčítačka:**

- Vezme se běžná vícebitová sčítačka (např. typu CLA).
- Vstup A jde přímo do sčítačky.
- Vstup B se nepřipojí přímo, ale každý jeho bit projde skrze hradlo XOR, které je řízeno řídicím signálem SUB (Subtract).
- Výstupní signál SUB je zároveň zaveden jako počáteční přenos C_{in} do úplné sčítačky nejnižšího bitu.
- **Režim Sčítání (SUB = 0):**
 - Hradla XOR propustí bity B nezměněné ($B \oplus 0 = B$).
 - Do nejnižšího přenosu C_{in} jde 0.
 - Obvod provede čisté sčítání $A + B$.
- **Režim Odčítání (SUB = 1):**
 - Hradla XOR zafungují jako invertory a znegují všechny bity čísla B ($B \oplus 1 = \bar{B}$).
 - Zároveň se do nejnižšího C_{in} přivede 1.
 - Sčítačka pak spočítá $A + \bar{B} + 1$, což je přesně definice odčítání v doplňkovém kódu.

D. Násobičky (Multipliers)

Násobení v binární soustavě probíhá na stejném principu jako ruční násobení na papíře: generují se částečné součiny a ty se následně sečtou se správným posunem. V binární soustavě je to snazší, protože násobíme buď 1 (částečný součin je roven násobenci), nebo 0 (částečný součin je 0).

- **Možnosti realizace:**

1. **Sériová násobička (Algoritmus posunu a sčítání):**

- **Konstrukce:** Využívá jeden registr pro násobenec, posuvný registr pro násobitele a jednu běžnou sčítačku.
- **Princip:** V každém kroku se kontroluje nejnižší bit násobitele. Pokud je 1, násobenec se přičte k akumulátoru. Násobenec se posune doleva a násobitel doprava.
- **Výhoda:** Je prostorově nenáročná (vyžaduje méně hradel).
- **Nevýhoda:** Je pomalá. Operace trvá n hodinových taktů pro n -bitové násobení.

2. **Paralelní (Pole) násobička (Array Multiplier):**

- **Konstrukce:** Čistě kombinační obvod. Skládá se z matice hradel AND (která generují částečné součiny) a sítě sčítaček (např. Wallaceův strom nebo pole úplných sčítaček), které tyto součiny bleskově sečtou naráz.
- **Výhoda:** Je extrémně rychlá (výpočet proběhne v rámci jednoho taktu).
- **Nevýhoda:** Spotřebovává obrovské množství křemíku na čipu, což vede k vysoké hardwarové náročnosti a spotřebě energie.

E. Detekce přetečení (Overflow)

Při aritmetických operacích může dojít k situaci, kdy se výsledek nevejde do vymezeného počtu bitů. Tomuto jevu se říká přetečení (Overflow). Procesor na to musí reagovat nastavením příznaku (Flag) ve stavovém registru.

- **U neznaménkových čísel:**

- K přetečení dojde, pokud vznikne přenos z nejvyššího bitu sčítačky (C_{out} z MSB je rovno 1).
- Jednoduše: výsledek je větší než maximální hodnota, kterou lze n bity reprezentovat.

- **U znaménkových čísel (v doplňkovém kódu):**

- Přenos z nejvyššího bitu nemusí znamenat chybu.
- Přetečení zde nastává tehdy, pokud sečteme dvě čísla se stejným znaménkem a dostaneme výsledek se znaménkem opačným (např. kladné + kladné = záporné, nebo záporné + záporné = kladné).

- **Hardwarová detekce pro znaménková čísla:**

- Provádí se porovnáním přenosu *do* nejvyššího řádu (C_{in_MSB}) a přenosu z nejvyššího řádu *ven* (C_{out_MSB}) pomocí hradla XOR.
- Pokud se tyto dva přenosy liší, nastalo přetečení:

$$\text{Overflow Flag} = C_{in_MSB} \oplus C_{out_MSB}$$

Praktické použití

- **Procesory (CPU):** Základní stavební bloky pro veškeré aritmetické operace v CPU.
 - **Mikrořadiče:** Pro jednoduché výpočty v embedded systémech.
 - **Digitální signálové procesory (DSP):** Optimalizované pro rychlé aritmetické operace, zejména násobení a sčítání, pro zpracování zvuku, obrazu atd.
 - **FPGA akcelerátory:** Pro implementaci vysoce paralelních a specializovaných aritmetických jednotek, kde je kritická rychlost a propustnost.
 - **Grafické procesory (GPU):** Obsahují tisíce aritmetických jednotek pro paralelní výpočty v grafice a strojovém učení.
-

Nejčastější chyby u zkoušky

- **Nezohlednění přenosu mezi bity:** Zapomenutí na C_{in} u úplné sčítačky nebo C_{out} při řetězení.

- **Záměna přetečení u signed a unsigned čísel:** Použití stejné logiky detekce přetečení pro oba typy čísel.
 - **Představa, že odečítání vyžaduje zcela jiný obvod než sčítání:** Neznalost principu odčítání v doplňkovém kódu pomocí sčítačky.
 - **Nesprávné pochopení časové složitosti CLA sčítačky:** Tvzení, že je striktně $O(1)$ pro libovolně velký počet bitů, namísto $O(\log n)$ nebo $O(1)$ pro bloky.
 - **Neznalost kompromisů mezi sériovou a paralelní násobičkou:** Neumět vysvětlit trade-off mezi rychlostí a hardwarovou náročností.
-

Paměťová pomůcka

Derivace říká "jak rychle se mění", integrál říká "kolik se nahromadilo".

Aritmetické obvody: Sčítačka je základ, dvojkový doplněk zjednodušuje, přetečení je past.

Typické zkouškové otázky

1. **Vysvětlete, proč se v moderních počítačích používá dvojkový doplněk pro reprezentaci záporných čísel.**
 - **Odpověď:** Dvojkový doplněk se používá, protože řeší dva hlavní problémy jiných reprezentací:
 1. **Jednoznačná reprezentace nuly:** Na rozdíl od přímého a jedničkového kódu má dvojkový doplněk pouze jednu reprezentaci nuly (00...0).
 2. **Zjednodušení aritmetiky:** Umožňuje provádět sčítání i odčítání pomocí téže sčítačky. Odčítání $A - B$ se převede na sčítání $A + (-B)$, kde $-B$ je dvojkový doplněk čísla B . Tím se výrazně zjednodušuje návrh hardwaru.
2. **Popište rozdíl mezi sčítačkou s řetězovým přenosem (RCA) a sčítačkou s předpovědí přenosu (CLA) z hlediska rychlosti a složitosti.**

○ **Odpověď:**

- **Sčítačka s řetězovým přenosem (RCA):** Je konstruována sériovým zapojením úplných sčítaček, kde přenos z nižšího řádu se šíří postupně do vyšších řádů. Její **rychlost** je omezená, protože výpočet každého bitu závisí na přenosu z předchozího bitu, což vede k lineárnímu zpoždění s počtem bitů ($O(n)$). Její **složitost** je relativně nízká, skládá se z jednoduchého opakování úplných sčítaček.
- **Sčítačka s předpovědí přenosu (CLA):** Používá speciální kombinační logiku k paralelnímu výpočtu všech přenosů současně. Tím výrazně zvyšuje **rychlost**, protože eliminuje řetězové zpoždění přenosu. Její časová složitost je výrazně lepší, často se považuje za $O(\log n)$ nebo téměř $O(1)$ pro praktické bloky. Její **složitost** je však podstatně vyšší, vyžaduje více hradel a komplexnější zapojení, což zvyšuje hardwarovou náročnost.

3. **Jak lze realizovat odčítání $A - B$ pomocí sčítačky v doplňkovém kódu? Uveďte logickou rovnici, která se pro to využívá.**

- **Odpověď:** Odčítání $A - B$ se v doplňkovém kódu realizuje jako sčítání $A + (-B)$. Číslo $-B$ se získá tak, že se nejprve provede bitová negace všech bitů čísla B (jedničkový doplněk, $\bar{B}$) a k tomuto výsledku se přičte jednička.
- Logická rovnice pro odčítání pomocí sčítačky je:

$$A - B = A + \bar{B} + 1$$

- Hardwarově se to provede tak, že bity čísla B projdou hradly XOR, které jsou řízeny signálem SUB . Pokud je $SUB = 1$, bity B se invertují ($\bar{B}$). Tento signál $SUB = 1$ se zároveň přivede jako počáteční přenos (C_{in}) do nejnižšího bitu sčítačky, čímž se efektivně přičte ta jednička.

4. **Vysvětlete princip detekce přetečení (overflow) pro znaménková čísla v doplňkovém kódu.**

- **Odpověď:** Přetečení u znaménkových čísel v doplňkovém kódu nastává, když výsledek aritmetické operace překročí rozsah reprezentovatelných hodnot pro daný počet bitů a změní se znaménko výsledku. Konkrétně k němu dojde, pokud sečteme dvě kladná čísla a výsledek je záporný, nebo sečteme dvě záporná čísla a výsledek je kladný.
- Hardwarově se přetečení detekuje porovnáním přenosu *do* nejvyššího bitu (MSB) a přenosu *z* nejvyššího bitu (MSB). Pokud se tyto dva přenosy liší, došlo k přetečení.
- Formálně se příznak přetečení (Overflow Flag) vypočítá pomocí hradla XOR:

$$\text{Overflow Flag} = C_{in_{MSB}} \oplus C_{out_{MSB}}$$

5. Porovnejte sériovou a paralelní násobičku z hlediska rychlosti a hardwarové náročnosti.

- **Odpověď:**
 - **Sériová násobička:**
 - **Rychlost:** Je pomalá, protože násobení n -bitových čísel vyžaduje n hodinových taktů. Operace probíhá sekvenčně.
 - **Hardwarová náročnost:** Je nízká. Využívá méně hradel a registrů, což šetří místo na čipu a snižuje spotřebu energie.
 - **Paralelní násobička (Array Multiplier):**
 - **Rychlost:** Je velmi rychlá, protože všechny částečné součiny jsou generovány a sčítány paralelně v rámci jednoho hodinového taktu.
 - **Hardwarová náročnost:** Je vysoká. Vyžaduje velký počet hradel (matice hradel AND a sít' sčítaček), což vede k většímu prostoru na čipu a vyšší spotřebě energie.
 - Volba mezi nimi závisí na požadavcích aplikace: sériová pro úsporné a méně kritické aplikace, paralelní pro vysoce výkonné systémy.
-

Shrnutí kapitoly

Aritmetické obvody jsou základem digitálních počítačů. Správná reprezentace čísel, zejména dvojkový doplněk pro znaménková čísla, zjednodušuje návrh sčítaček a odčítaček. Sčítačky se dělí na pomalé s řetězovým přenosem a rychlé s předpovědí přenosu. Odčítání se efektivně realizuje pomocí sčítaček a dvojkového doplnku. Násobičky existují v sériových (pomalých, úsporných) a paralelních (rychlých, náročných) variantách. Klíčové je také pochopení a detekce přetečení, které se liší pro znaménková a neznaménková čísla. Tyto obvody tvoří základ pro ALU (Arithmetic Logic Unit) v procesorech.

Níže je připravený studijní materiál k okruhu "Procesory, FPGA obvody" pro státní závěrečné zkoušky, doplněný a naformátovaný dle zadaných přísných pravidel.

37. Procesory, FPGA obvody

Klíčové pojmy

- procesor (CPU, MCU)
- instrukční soubor
- registr (Program Counter - PC, Stack Pointer - SP)
- ALU (Aritmeticko-logická jednotka)
- pipeline
- cache
- přerušení
- konečný automat (FSM)
- FPGA (Field Programmable Gate Array)
- CLB (Configurable Logic Block) / LE (Logic Element)
- LUT (Look-Up Table)
- klopný obvod (D-Flip-Flop)
- propojovací matice
- I/O bloky
- HDL (Hardware Description Language)
- VHDL
- Verilog
- syntéza
- bitstream
- paralelismus
- SoC (System on Chip)

A. Procesory (CPU / MCU)

Definice a princip

Procesory (Central Processing Unit – CPU nebo jednočipové mikrokontroléry – MCU) jsou univerzální programovatelné sekvenční obvody. Jejich základním principem je sekvenční čtení strojových instrukcí z paměti a jejich následné provádění. Jsou flexibilní a vhodné pro širokou škálu úloh.

Architektura procesoru

Srdcem procesoru je **řadič (Control Unit)**, který je sám o sobě velmi složitým, pevně zadrátovaným (nebo mikroprogramovaným) konečným automatem. Tento "metautomat" v nekonečné smyčce provádí základní **Fetch-Decode-Execute cyklus**:

- **Fetch (Načtení):** Instrukce je načtena z paměti na adrese uložené v registru **PC (Program Counter)**. Po načtení se hodnota PC obvykle inkrementuje na další instrukci.
- **Decode (Dekódování):** Načtená instrukce je dekována, aby procesor zjistil, o jakou operaci se jedná (např. sčítání, přesun dat, skok) a jaké operandy (registry, paměťové adresy) jsou potřeba.
- **Execute (Vykonání):** Operace je provedena. To může zahrnovat povel pro **ALU (Aritmeticko-logická jednotka)** k sečtení hodnot v registrech, zápis dat do paměti, změnu hodnoty PC pro skok, nebo I/O operaci.

Moderní procesory využívají techniky jako **pipeline** (paralelní zpracování různých fází instrukcí) a **cache** (rychlá paměť pro často používaná data a instrukce) pro zvýšení výkonu.

Realizace konečných automatů (FSM) pomocí procesoru

Pokud potřebujeme pomocí mikrokontroléru (např. v embedded systémech) realizovat konečný automat (např. řízení křižovatky, parsování textu), děláme to čistě softwarově.

- **Implementace v kódu:**
 - Stav automatu je uložen v proměnné (registru nebo RAM).

- Přejchodová logika a výstupy jsou realizovány sekvenčním kódem, nejčastěji pomocí konstrukce `switch-case` uvnitř nekonečné smyčky.
- Příklad (z materiálu):

```
c switch (state) { case S0: if (input) state = S1; break; case S1: state = S0; break; }
```
- **Charakteristika:** Výpočet je striktně sekvenční. Procesor nemůže vyhodnotit všechny podmínky naráz. Musí instrukci po instrukci otestovat vstupy, změnit stav v paměti a zapsat výstup na registry I/O portů. To vede k časové režii a omezenému paralelismu.

Praktické použití procesorů

- **Obecné aplikace:** Běžné počítače, servery, mobilní telefony.
- **Operační systémy:** Základní platforma pro běh OS.
- **Embedded systémy:** Mikrokontroléry v řídicích jednotkách, IoT zařízeních.

Nejčastější chyby u zkoušky

- Ignorování konceptů jako pipeline a cache při popisu výkonu procesoru.
- Představa, že procesor může provádět více operací *současně* v pravém slova smyslu (bez ohledu na jádra, pipeline atd. – v rámci jednoho vlákna je sekvenční).

B. FPGA obvody (Field Programmable Gate Array)

Definice a princip

FPGA (Field Programmable Gate Array) je polovodičová součástka, která po výrobě neobsahuje žádný pevný procesor ani konkrétní logickou funkci. Skládá se z obrovského množství univerzálních digitálních bloků, jejichž vnitřní funkčnost a vzájemné propojení lze konfigurovat (naprogramovat) až u uživatele. Umožňuje implementovat vlastní, vysoce paralelní hardware.

Vnitřní architektura FPGA

Základní struktura se skládá ze tří hlavních prvků:

- **CLB / LE (Configurable Logic Blocks / Logic Elements):** Základní stavební buňky. Každý blok obsahuje:
 - **LUT (Look-Up Table):** Malá statická paměť (RAM), která funguje jako univerzální kombinační obvod. Pokud chceme realizovat logickou funkci s N vstupy, pravdivostní tabulka se nahraje do LUT. Vstupy pak fungují jako adresové vodiče a LUT okamžitě vrací výsledek (podobně jako multiplexor).
 - **Klopný obvod (D-Flip-Flop):** Paměťový prvek pro sekvenční logiku. Výstup z LUT lze poslat buď přímo ven (kombinační režim), nebo ho nechat vzorkovat hranou hodinového signálu v klopném obvodu (sekvenční režim), čímž se uloží stav.
 - **Sčítáčková logika (Carry chains):** Dedikovaný hardware pro rychlé sčítání a přenosy, optimalizovaný pro aritmetické operace.
- **Propojovací matice (Programmable Routing):** Síť horizontálních a vertikálních vodičů s programovatelnými spínači, která umožňuje propojit jakýkoliv logický blok s jakýmkoliv jiným. Tato flexibilita je klíčová pro rekonfiguraci.
- **I/O Bloky (Input/Output Blocks):** Obvodové buňky zajišťující rozhraní mezi vnitřní logikou FPGA a fyzickými piny čipu, umožňující komunikaci s vnějším světem.
- **DSP bloky a paměťové bloky:** Moderní FPGA často obsahují i dedikované bloky pro digitální zpracování signálů (DSP) a blokové RAM paměti pro efektivnější implementaci složitějších funkcí.

Realizace konečných automatů (FSM) pomocí FPGA

Na rozdíl od procesoru se v FPGA automat nerealizuje softwarovým programem. Automat se v FPGA fyzicky vytvoří z hardwarových struktur.

- **LUT tabulky** se nakonfigurují tak, aby tvořily přechodovou a výstupní kombinační logiku automatu.
- **Klopné obvody** uvnitř logických bloků fyzicky drží zakódovaný vnitřní stav automatu.

- **Charakteristika:** Výpočet probíhá čistě hardwarově a masivně paralelně. Všechny přechodové podmínky a výstupy se vyhodnocují současně v reálném čase, v závislosti na hraně hodinového signálu s minimálním zpožděním. To umožňuje dosáhnout velmi vysokého výkonu a nízké latence pro specifické úlohy.

Praktické použití FPGA

- **Zpracování signálů:** Audio, video, telekomunikace (filtrace, modulace/demodulace).
- **Měření a řízení:** Průmyslové řídicí systémy, akvizice dat.
- **Akcelerátory:** Kryptografie, počítačové vidění, strojové učení.
- **Prototypování ASIC:** Rychlé ověření návrhu před výrobou specializovaného čipu.

Nejčastější chyby u zkoušky

- Tvzení, že FPGA "spouští program" stejně jako CPU. FPGA *implementuje* hardware, ne spouští program.
- Ignorování hodinových signálů a synchronizace, které jsou pro sekvenční obvody ve FPGA klíčové.
- Zaměňování HDL (Hardware Description Language) s běžným programovacím jazykem.

C. Srovnání realizace automatů: Procesor vs. FPGA

Vlastnost	Procesor (CPU / MCU)	FPGA (Field Programmable Gate Array)
Povaha	Univerzální instrukční stroj	Rekonfigurovatelný hardware
Princip činnosti	Sekvenční vykonávání instrukcí (software)	Paralelní implementace logiky (hardware)

Vlastnost	Procesor (CPU / MCU)	FPGA (Field Programmable Gate Array)
Realizace FSM	Softwarová (proměnná pro stav, switch-case)	Hardwarová (LUT pro logiku, klopné obvody pro stav)
Paralelismus	Omezený (pipeline, více jader), sekvenční instrukce	Masivní paralelismus, skutečně souběžné operace
Latence	Vyšší (režie OS, instrukční cykly)	Velmi nízká (přímá hardwarová implementace)
Flexibilita	Vysoká (snadná změna chování úpravou programu)	Střední (změna vyžaduje novou syntézu a implementaci)
Vývoj	Jednodušší (běžné programovací jazyky)	Složitější (HDL, časová omezení, simulace)
Spotřeba energie	Nižší pro obecné úlohy, vyšší pro specifické úlohy	Vyšší pro obecné úlohy, nižší pro specifické úlohy
Cena	Nižší pro sériovou výrobu, vyšší pro prototypování	Vyšší pro sériovou výrobu, nižší pro prototypování
Použití	Operační systémy, obecné aplikace, servery	Zpracování signálů, akcelerátory, řízení, embedded

D. Jazyky pro popis hardwaru (HDL)

Když programujeme FPGA, nepíšeme program, který se bude "spouštět". Píšeme kód, který definuje, jak se mají hradla a paměťové prvky propojit a

chovat. K tomu slouží **jazyky pro popis hardwaru (HDL)**, jako jsou **VHDL** nebo **Verilog**.

Kompilační proces pro FPGA

Kompilační proces pro FPGA se diametrálně liší od běžného překladu softwaru:

1. **Syntéza:** HDL kód se přeloží do sítě konkrétních logických hradel a klopných obvodů (tzv. **Netlist**). V této fázi se abstraktní popis chování převádí na konkrétní logické obvody.
2. **Implementation (Place & Route):** Speciální algoritmus rozhodne, které konkrétní logické bloky (LE) na čipu se pro implementaci použijí (**Placement**) a jakými vodiči se v propojovací matici fyzicky propojí (**Routing**). Cílem je splnit časová omezení (timing constraints) a minimalizovat zpoždění.
3. **Generování bitstreamu:** Vytvoří se konfigurační soubor (bitstream), který se nahraje do statické paměti FPGA při startu a fyzicky sepne spínače v obvodu, čímž se vytvoří požadovaný hardware.

Nejčastější chyby u zkoušky

- Zaměňování HDL s běžným programovacím jazykem (např. C++). HDL popisuje *hardware*, ne sekvenční *program*.
- Ignorování časování a kritické cesty. Pro FPGA je klíčové, aby signály dorazily včas a obvod pracoval na požadované frekvenci.

E. Koncept SoC (System on Chip)

Dnešní moderní architektury tyto dva světy propojují. Vznikají čipy, kde je na jednom křemíku vedle sebe umístěno výkonné pevné jádro procesoru (např. ARM) a oblast programovatelného hradlového pole (FPGA).

Definice a výhody

SoC (System on Chip) je integrovaný obvod, který integruje všechny nebo většinu komponent počítače nebo jiného elektronického systému do

jediného čipu. Kombinace CPU a FPGA na jednom SoC nabízí to nejlepší z obou světů:

- **Flexibilita procesoru:** Pro řešení složité, ale na rychlost méně kritickou logiku (např. operační systém, síťovou komunikaci, uživatelské rozhraní).
- **Výkon a paralelismus FPGA:** Pro implementaci dedikovaných automatů pro ultra-rychlé zpracování signálů, šifrování, počítačové vidění nebo jiné hardwarové akcelerátory.

Příklady použití

- **Vestavěné systémy:** Pokročilé řídicí jednotky, autonomní vozidla.
- **Síťová zařízení:** Routery, přepínače s vysokým výkonem.
- **Zpracování obrazu a videa:** Real-time analýza, kódování/dekódování.
- **Datová centra:** Akcelerace specifických úloh (např. AI inference).

Typické zkouškové otázky

1. Jaký je rozdíl mezi CPU a FPGA?

Odpověď: Hlavní rozdíl spočívá v jejich architektuře a způsobu vykonávání úloh:

- **CPU (procesor)** je univerzální sekvenční stroj, který vykonává instrukce uložené v paměti. Je flexibilní, ale jeho operace jsou sekvenční (i když moderní CPU používají pipeline a více jader pro zvýšení propustnosti, základní princip je sekvenční). Chování se mění změnou softwaru.
- **FPGA** je rekonfigurovatelný hardware. Místo vykonávání instrukcí se konfiguruje tak, aby fyzicky implementoval požadovanou logiku a obvody. To umožňuje masivní paralelismus a velmi nízkou latenci pro specifické úlohy, protože operace probíhají souběžně v hardwaru. Chování se mění rekonfigurací hardwaru (nahráním nového bitstreamu).

2. Co je pipeline a cache?

Odpověď:

- **Pipeline (instrukční roura)** je technika v architektuře procesorů, která umožňuje překrývání fází vykonávání instrukcí. Místo čekání na dokončení jedné instrukce, než se začne s další, se různé fáze (Fetch, Decode, Execute, Write-back) několika instrukcí provádějí souběžně v různých částech procesoru. Tím se zvyšuje propustnost procesoru, i když latence jednotlivé instrukce se nemusí nutně snížit.
- **Cache (vyrovnávací paměť)** je malá, velmi rychlá paměť umístěná blízko procesoru (často přímo na čipu). Slouží k ukládání kopií dat a instrukcí z hlavní paměti, které procesor pravděpodobně brzy znovu použije. Díky mnohem rychlejšímu přístupu k datům v cache oproti hlavní paměti se výrazně zkracuje doba čekání procesoru na data a zvyšuje se celkový výkon systému. Existují různé úrovně cache (L1, L2, L3).

3. Kdy zvolit FPGA místo procesoru?

Odpověď: FPGA je vhodnější volbou místo procesoru v situacích, které vyžadují:

- **Masivní paralelismus:** Pro úlohy, kde je potřeba provádět mnoho operací souběžně a nezávisle (např. zpracování obrazu, digitální zpracování signálů, kryptografie).
- **Velmi nízkou latenci a determinismus:** Pro aplikace, kde je kritické, aby odezva byla co nejrychlejší a předvídatelná (např. řízení v reálném čase, vysokofrekvenční obchodování).
- **Vysokou propustnost pro specifické úlohy:** Když sekvenční výkon procesoru není dostatečný pro objem dat nebo rychlost operací.
- **Hardwarovou akceleraci:** Pro specifické algoritmy, které lze efektivně namapovat na paralelní hardware a získat tak výrazné zrychlení.
- **Flexibilitu hardwaru po výrobě:** Možnost změnit hardwarovou funkci čipu i po jeho nasazení, což je výhodné pro standardy, které se vyvíjejí, nebo pro opravy chyb v hardwaru.

Procesor je naopak vhodnější pro obecné výpočetní úlohy, kde je klíčová flexibilita softwaru, snadnost programování, rychlý vývoj a kde sekvenční povaha výpočtu není kritickým omezením.

Shrnutí kapitoly

Procesory jsou flexibilní univerzální stroje, které vykonávají instrukce sekvenčně. FPGA obvody umožňují vytvořit specializovaný paralelní hardware přímo na čipu. Volba mezi procesorem a FPGA závisí na požadavcích na výkon, latenci, paralelismus, flexibilitu, cenu a náročnost vývoje. Koncept SoC kombinuje výhody obou přístupů na jednom čipu.
